

2.4 — Integrate MCP Servers into Claude Code and Agent Workflows

Concept

What is `.mcp.json`?

`.mcp.json` is the primary configuration file for registering MCP servers with Claude Code. It declares which servers are available, how to launch them, and what environment they receive. Claude Code reads this file at startup and establishes connections to each listed server before the session begins.

Project Scope vs User Scope

MCP servers can be registered at two levels of scope:

Project scope — `.mcp.json` placed in the root of a project repository. This file is checked into version control and shared across everyone working on that project. Project-scope servers are only active when Claude Code is opened inside that directory tree.

User scope — configuration stored in `~/.claude/mcp.json` (or `~/.claude/settings.json` under the `mcpServers` key). User-scope servers are available globally across all Claude Code sessions, regardless of which project is open. This is the right place for personal tooling such as a local search server or a private data connector.

When both scopes define a server with the same name, project scope takes precedence. This allows teams to override a user-global server with a project-specific variant without touching the user's home directory.

Transport: stdio vs SSE

Claude Code supports two transports for MCP communication:

- **stdio** — Claude Code spawns the server as a child process and communicates over stdin/stdout. This is the standard choice for local servers; it is simple, requires no networking, and the server's lifetime is bound to the Claude Code session.
- **SSE (Server-Sent Events)** — Claude Code connects to an already-running HTTP server via a URL. SSE is appropriate for shared or remote servers, or when the server must persist between sessions (e.g., a team-wide database connector running in a container).

The transport is specified per server in `.mcp.json`.

Environment Variable Expansion

Hard-coding secrets in `.mcp.json` is a security risk and breaks portability. Claude Code supports the `env://` prefix to pull values from the shell environment at connection time:

```
"env": {
  "API_KEY": "env://MY_SERVICE_API_KEY"
}
```

At startup, `env://MY_SERVICE_API_KEY` is resolved to the value of `$MY_SERVICE_API_KEY` in the current shell environment. If the variable is not set, Claude Code surfaces an error rather than silently passing an empty string, preventing hard-to-debug failures.

MCP Resources and Resource Templates

Beyond tools, MCP servers can expose **resources** — structured, addressable content that Claude can read as context. Resources have a URI scheme specific to the server (e.g., `db://customers/42` or `file://reports/q4.pdf`).

Static resources are declared with a fixed URI and a known MIME type. Claude Code can list them and fetch their content on demand.

Resource templates use URI templates (RFC 6570) to parameterize access:

```
customers://{id}/profile
```

When Claude needs customer 42's profile, it resolves the template to `customers://42/profile` and fetches it. Templates let a single declaration cover an unbounded set of resources without enumerating each one.

Resources differ from tool calls in an important way: they are read-only and intended for context injection, not side-effecting actions. This distinction matters for permission models and audit trails.

Code Example

Project-scope `.mcp.json` with stdio and SSE servers:

```
{
  "mcpServers": {
    "local-db": {
      "type": "stdio",
      "command": "node",
      "args": ["/tools/db-server/index.js"],
      "env": {
        "DATABASE_URL": "env://DATABASE_URL",
        "DB_API_KEY": "env://DB_API_KEY"
      }
    },
    "shared-search": {
      "type": "sse",
      "url": "https://search.internal.example.com/mcp/sse",
      "env": {
```

```

    "SEARCH_TOKEN": "env://SEARCH_TOKEN"
  }
}
}
}

```

TypeScript MCP server exposing a tool and a resource template:

```

import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

const server = new McpServer({
  name: "customer-server",
  version: "1.0.0",
});

// Tool: side-effecting action
server.tool(
  "create_customer",
  { name: z.string(), email: z.string().email() },
  async ({ name, email }) => {
    const id = await db.customers.create({ name, email });
    return { content: [{ type: "text", text: `Created customer ${id}` }] };
  }
);

// Resource template: read-only context
server.resource(
  "customer-profile",
  new ResourceTemplate("customers://{id}/profile", { list: undefined }),
  async (uri, { id }) => {
    const customer = await db.customers.findById(id);
    return {
      contents: [{
        uri: uri.href,
        mimeType: "application/json",
        text: JSON.stringify(customer),
      }],
    };
  }
);

const transport = new StdioServerTransport();
await server.connect(transport);

```

Common Mistakes

Committing secrets to .mcp.json. Always use `env://` references. A `.mcp.json` with a literal API key will be checked into version control and exposed to every contributor and CI system.

Misidentifying scope. Registering a project-specific server in user scope (`~/.claude/mcp.json`) means it runs even outside the project — wasting resources and potentially exposing internal tooling in unrelated sessions.

Conflating tools and resources. Using a tool to return large read-only documents bloats the tool call history and consumes output tokens unnecessarily. Expose large context (documentation, database records, file contents) as resources and use tools only for actions with side effects.

SSE server not running at startup. Unlike stdio, SSE servers must be reachable when Claude Code starts the session. A misconfigured or offline SSE endpoint causes the entire server registration to fail, which silently removes all its tools from the session without a clear error unless logs are checked.

Missing type field in .mcp.json. Omitting `"type": "stdio"` or `"type": "sse"` causes Claude Code to reject the entry. Both fields are required; there is no default transport.

References

- Model Context Protocol specification — Resources: <https://spec.modelcontextprotocol.io/specification/server/resources/>
- Claude Code MCP documentation: <https://docs.anthropic.com/en/docs/claude-code/mcp>
- MCP TypeScript SDK: <https://github.com/modelcontextprotocol/typescript-sdk>
- RFC 6570 URI Templates: <https://www.rfc-editor.org/rfc/rfc6570>